

Optimization for Machine Learning

Exercise sheet **Comparison of Sub-Gradient and Coordinate ascent (descent) Algorithms: Sub-Gradient, Min-Sum Diffusion, TRW-S and ICM.**

Bogdan Savchynskyy - Bogdan.Savchynskyy@tu-dresden.de
Alexander Kirillov - Alexander.Kirillov@tu-dresden.de

Algorithms In this exercise we use 5 different algorithms:

- **ICM** works in the primal domain with the non-relaxed energy function. It fixes labels assigned to all nodes except one and optimizes over all possible labels in this node. The process is repeated iteratively until none of the labels change.
- **Min-Sum Diffusion** performs block-coordinate ascent in the dual domain and optimizes the *Lagrange dual* over all possible reparametrization. Due to non-smoothness and non-strong convexity it may get stuck in suboptimal points, satisfying arc-consistency. Unfortunately, there is no implementation of this algorithm in the **OpenGM** library, therefore we use its surrogate having similar properties (but some-what faster convergence). This surrogate is the **TRW-S** algorithm with decomposition to trivial sub-trees, having one edge only. The dedicated parameters of the command line are **-a TRWSi -d EDGE** (see examples below).
- **TRW-S** performs the block-coordinate ascent in the dual domain and optimizes the *Lagrangian decomposition based dual*. Due to non-smoothness and non-strong convexity it may get stuck in suboptimal points, satisfying tree-agreement, which is equivalent to the arc-consistency. Different decompositions into monotonic sequences can be used with *TRW-S*. Typically these monotonic sequences are selected in a greedy fashion and de facto, need not even be computed explicitly.
- **Sub-gradient** performs the super-gradient steps in the dual domain and optimizes the *Lagrangian decomposition based dual*. In the theory, this algorithm converges to an optimum of the dual. In practice its behavior heavily depends on the selected step-size strategy. The strategy leading to the fastest convergence may change a lot from one problem instance to another one. The current **OpenGM** implementation of this algorithm allows to select different decompositions and step-size strategies. It is recommended to use spanning trees as a decomposition and tune the step-size parameters, which determine it according to one of the formulas below ?. (i) The **primal-dual-gap** is the difference between the energy of the current rounded integer solution and the dual objective; (ii) i is the iteration counter; (iii) e is the exponent of the step-size rule, the algorithm's parameter; (iv) s and α are the constants, "stride" and "scale", which can be selected manually as well.

1. Projected Adaptive step-size:

$$\frac{\text{primal-dual-gap}}{\|\text{projected-sub-gradient}\|^2(1 + (\alpha * i)^e)} \quad (1)$$

2. Step-Length Rule:

$$\frac{s}{\|\text{projected-sub-gradient}\|^2(1 + (\alpha * i)^e)} \quad (2)$$

3. Step-Size Rule:

$$\frac{s}{(1 + (\alpha * i)^e)} \quad (3)$$

The **projected sub-gradient** is a normal sub-gradient in the terminology of the lectures.

- **ADSa1** (Adaptive Diminishing Smoothing algorithm ?, not covered by the lectures) – the modification of TRW-S, which approximates the *Lagrangian decomposition based dual* objective with a smooth concave function prior to optimization. Due to the smoothing the block-coordinate ascent algorithms converge to the optimum of the smoothed objective. ADSa1 gradually decreases the smoothing as the iterates of the algorithm get closer to the optimum. This ensures convergence to an optimum of the initial non-smooth objective. Besides the guaranteed convergence, the algorithm provides an approximate feasible relaxed primal variable estimates ($\mu \in \mathcal{L}$) as an optimality certificate.

Exercise 1 (ICM vs. Diffusion). Download and unpack **color-seg-n4** datasets from the OpenGM benchmark <http://hci.iwr.uni-heidelberg.de/opengm2/> (or use the downloaded ones from the previous exercises). Use the python scripts to the previous exercise.

ICM Run the ICM solver on any dataset (e.g. **lake**, **crops**, **pfau**,...). Since the algorithm makes a lot of very small updates (for each node) output only each 100th update:

```
opengm_min_sum -a ICM -p 100 -v -o crops-icm.h5 -m crops-small.h5
```

Use the naïve solver to obtain the naïve value and bound for all datasets, e.g.:

```
python naive_solver.py crops-small.h5
```

Compare the naïve value and bound to the value of returned by the ICM.

Diffusion Run the diffusion solver on the same dataset (e.g. **lake**, **crops**, **pfau**,...). The solver's implementation is quite slow, however it is recommended to run it at least 100 iterations.

```
opengm_min_sum -a TRWSi -p 1 -v --maxIt 100 -d EDGE -o crops-dif.h5 -m crops-small.h5
```

Check that already after the first iteration the results obtained by the diffusion algorithm are better than the final ones of the ICM. Run the **diffusion** again at least on the **crops** and **lake** datasets until the absolute duality gap is less than 0.05 :

```
opengm_min_sum -a TRWSi -p 1 -v --maxIt 1000 -d EDGE -o crops-dif.h5 -m crops-small.h5 --precision 0.05
opengm_min_sum -a TRWSi -p 1 -v --maxIt 1000 -d EDGE -o lake-dif.h5 -m lake-small.h5 --precision 0.05
```

Compare the obtained values/bounds to those obtained in the exercise **Dual Of LP Relaxation. Reparametrization..**

Stereo Reconstruction Repeat similar experiments with `tsu-gm` instance from the `mrf-stereo` dataset:

ICM Run:

```
opengm_min_sum -a ICM -p 100 -v -o tsu-icm.h5 -m tsu-gm.h5
python naive_solver.py tsu-gm.h5
```

Diffusion Run:

```
opengm_min_sum -a TRWSi -p 1 -v --maxIt 100 -d EDGE -o tsu-dif.h5 -m tsu-gm.h5
```

Run diffusion until the absolute duality gap is less than 0.05 :

```
opengm_min_sum -a TRWSi -p 1 -v --maxIt 1000 -d EDGE -o tsu-dif.h5 -m tsu-gm.h5 --precision 0.05
```

Check how the results differ visually:

```
python show_labeling.py -V 288 -H 384 -v -C -f tsu-icm.h5
python show_labeling.py -V 288 -H 384 -v -C -f tsu-dif.h5
```

Exercise 2 (Sub-Gradient vs. TRW-S).

Synthetic problem Download the synthetic dataset from the supplementary material.
Run

```
./opengm_min_sum --modelinfo -m synthetic200x100x5_amp1.0_s1001.0.h5
```

What is your opinion, which what kind of structure has the graph of this model? Chain? Grid? Is it acyclic? How many labels it has?

The potentials of this model were generated randomly from the uniform distribution on the interval $[0, 1]$. The corresponding energy minimization problem is not LP tight. Moreover, even attaining an optimum of the dual LP problem is not easy, at least for coordinate descent algorithms like `diffusion` or `TRW-S` - they converge to arc-consistency, which is only necessary, but not sufficient for optimality.

In contrast, the `subgradient` method converges to the dual LP optimum, at least in theory. The task of this exercise – empirically find the best step-size schedule for the `subgradient` algorithm.

Before playing with `subgradient`, find first an approximate value of the optimal dual LP. To this end run `ADSa1`, which provably converges to the optimum and, moreover, provides an approximate feasible relaxed primal solution as an optimality certificate.

```
./opengm_min_sum -a ADSa1 --maxIt 200 -m synthetic200x100x5_amp1.0_s1001.0.h5 --precision 50 -p 1 -v
```

The energy of the estimated primal relaxed solution is shown as `primalLPbound`.

Run `TRW-S` to get an idea of its convergence.

```
./opengm_min_sum -a TRWS --maxIt 200 -m synthetic200x100x5_amp1.0_s1001.0.h5 -p 1 -v
```

Run the `subgradient` algorithm. If you see that it has numerically got stuck, stop it by pressing `Ctrl-C`:

```
./opengm_min_sum -a DSubgradient --maxIt 200 -m synthetic200x100x5_amp1.0_s1001.0.h5 -p 1 -v
```

Check parameters of the `subgradient` algorithm, which define its used step-size:

```
./opengm_min_sum -a DSubgradient -h
```

Consider playing with 3 different step-size rules: `ProjectedAdaptive`, `StepLength`, `StepSize`. In general they depend on:

1. `primalDualGap = value-bound`;
2. `a` - scale of the step-size rule, is regulated by the parameter `--ssScale`;
3. `i` -iteration counter;
4. `e` -exponent of the step-size (e.g. the default square root value is obtained by setting `e= 0.5`), is regulated by the parameter `--ssExponent`;
5. you might also consider to play with the minimal and maximal step-sizes (parameters `--ssMin` and `--ssMax`) and decomposition type (parameter `--decomp`).

Play with parameters with the goal to attain better dual LP bound than those of `TRW-S`.

Color segmentation Run the `subgradient` method on `crops` and `clownfish`. First do it with the best parameters from the previous exercise. Afterwards try to tune parameters to the given datasets. Compare the obtained iteration-wise convergence speed to those of the `TRW-S` algorithm.

Stereo Run the `subgradient` method on `tsu-gm` and `ven-gm`. First do it with the best parameters from the previous exercises. Afterwards try to tune parameters to the given datasets. Compare the obtained iteration-wise convergence speed to those of the `TRW-S` algorithm.